

ncsim: A Lightweight Discrete-Event Simulator for Networked Computing with 802.11 WiFi Models

Bhaskar Krishnamachari*, Maya Gutierrez*, and Jared Coleman*[†]

*University of Southern California

bkrishna@usc.edu, mayaguti@usc.edu

[†]Loyola Marymount University

jared.coleman@lmu.edu

Abstract—Evaluating DAG task schedulers for wireless edge computing requires jointly modeling compute placement and wireless interference, yet existing tools treat them in isolation. This gap leads to *rank inversions*: the scheduler that appears optimal under an interference-free model can be the worst choice under realistic wireless conditions. We present `ncsim`, a lightweight discrete-event simulator that bridges this gap by combining DAG workflow scheduling with physically-grounded IEEE 802.11 CSMA/CA interference modeling in a single Python package. A 108-run factorial experiment reveals rank inversions in 27.8% of scenarios, with the interference-free-optimal scheduler producing up to $2.7\times$ worse makespan than a simple round-robin baseline; scaling to a 100-node random geometric graph raises the inversion rate to 50%. Interference-free evaluation does not merely mis-predict performance but can reverse which algorithm a researcher would deploy, thereby justifying the design and use of `ncsim`.

I. INTRODUCTION

Edge and fog computing architectures distribute computation across heterogeneous nodes connected by networks of varying capacity and reliability [1]. Effective task scheduling in these environments requires co-optimizing compute placement and network transport: a task placement that minimizes computation time may incur excessive data transfer costs, while a network-aware placement may underutilize fast processors.

Existing simulation tools address parts of this problem. Network simulators such as ns-3 [2] and OMNeT++ [3] provide detailed protocol stacks but are heavyweight, complex to configure, and lack native support for DAG-based task scheduling. Conversely, task scheduling simulators such as SimGrid [4], WRENCH [5], and CloudSim [6] model workflows and compute resources but typically abstract the network as point-to-point links with fixed bandwidths, omitting wireless contention, interference, and multi-hop effects. Edge-specific tools like iFogSim [7] and EdgeCloudSim [8] add domain-specific features but use simplified wireless models that do not capture the distance-dependent rates, CSMA contention, or hidden terminal interference that characterize real 802.11 deployments.

This gap matters more than one might expect. In a 108-run factorial study, we find that the scheduler appearing optimal under an interference-free model is outperformed under realistic CSMA/CA conditions in 27.8% of scenarios, a *rank inversion*. In the most striking case, the HEFT scheduler produces $2.7\times$ worse makespan than a simple round-robin baseline,

yet an interference-free evaluation would confidently select HEFT. Realistic wireless modeling is essential for meaningful evaluation of edge computing scheduling algorithms.

We present `ncsim`,¹ a lightweight, flow-level discrete-event simulator that bridges this gap by combining DAG workflow scheduling with physically-grounded 802.11 WiFi models in a single Python package. The contributions are:

- 1) **ncsim simulator**: a modular architecture combining DAG scheduling (HEFT, CPOP, round-robin) with 802.11 WiFi models (log-distance path loss, MCS rate adaptation, Bianchi CSMA/CA contention, hidden terminal SINR degradation, multi-hop routing with dynamic bandwidth sharing) in $\sim 5,500$ lines of Python.
- 2) **Three-stage validation**: nine internal experiments matching analytical predictions exactly, reproduction of Bianchi’s published results [9], and cross-validation against ns-3 packet-level simulations (mean error 4%).
- 3) **Rank inversion finding**: a 108-run factorial on grid networks revealing that interference-free evaluation produces wrong scheduler rankings in 27.8% of scenarios.
- 4) **Scalability confirmation**: a 36-run study on a 100-node random geometric graph with 30–50 task DAGs, where the rank inversion rate rises to 50%.

Sections II to V survey related work and describe `ncsim`’s architecture, system model, and simulation engine. Section VI validates the WiFi model, and Section VII presents the experimental evaluation. Section VIII discusses implications and future directions.

II. RELATED WORK

We situate `ncsim` at the intersection of two research threads: (i) DAG scheduling algorithms and edge orchestration systems, and (ii) simulation tools that model wireless network effects. We survey both and highlight the gap that motivates our work. Table I compares simulation tools.

Classic DAG scheduling heuristics. DAG scheduling has a rich history in parallel computing, from list-scheduling [11], [12] and dynamic-level scheduling [13] to the widely used HEFT and CPOP heuristics [10]. These formulations assume stable platforms with known costs, a poor fit for wireless/edge deployments.

¹Open-source: <https://github.com/ANRGUSC/ncsim>

Benchmarking and parametric scheduler design. Comparative conclusions depend strongly on benchmark selection. SAGA and PISA [14], [15] enable adversarial search across schedulers, while parametric decomposition [16] supports systematic ablations across 72 hybrid variants. These tools do not model the wireless layer.

Dispersed and edge execution systems. Jupiter [17], Tactical Jupiter [18], and throughput-oriented extensions [19] integrate DAG scheduling with profiling and orchestration on real clusters. These efforts highlight that scheduling quality depends on profiling fidelity and network dynamics, but do not model wireless contention.

DAG scheduling in wireless/edge/IoT environments. A growing body of work targets edge environments with heterogeneous devices: I-BOT [20] addresses unmanaged edge devices, IBDASH [21] jointly optimizes latency and failure probability, and M-TEC [22] generalizes to multi-tier settings. Platform-oriented systems include LAVEA [23], Hetero-Edge [24], and Petrel [25]. MEC-oriented DAG offloading commonly models wireless links via abstract rates [26], [27], [28], and the broader fog/edge context is surveyed in [29], [30], [31]. Most of these works assume abstract network models that do not capture wireless contention at the physical layer.

Network simulators. ns-3 [2] and OMNeT++ [3] provide detailed WiFi, LTE, and TCP/IP stacks with high-fidelity channel models. However, they are heavyweight: configuring custom DAG scheduling requires external workflow engines and C++ glue code, and packet-level simulation times make large parameter sweeps prohibitive (see Table I).

Task scheduling simulators. SimGrid [4], WRENCH [5], and CloudSim [6] support DAG workflows natively but model networks as links with fixed or analytically-derived bandwidths. This abstraction fits wired data centers but omits wireless contention, hidden terminals, and spatial reuse, precisely the phenomena whose effective bandwidth depends on which other links are simultaneously active.

Edge/fog-specific simulators. iFogSim [7] and EdgeCloudSim [8] add edge-specific features (latency-sensitive placement, mobility) but use simplified wireless models that do not capture distance-dependent rates, CSMA/CA contention, or hidden terminal interference.

Positioning of ncsim. As Table I summarizes, ncsim uniquely combines DAG-aware scheduling with physically-grounded 802.11 wireless models in a lightweight Python package, providing a controlled simulation environment where wireless effects can be systematically varied and their impact on scheduling quality precisely measured.

III. ARCHITECTURE

In this section, we describe the architecture of ncsim, which is organized around a layered discrete-event simulation (DES) engine with pluggable components for scheduling, routing, and interference modeling. Figure 1 illustrates the data flow from scenario specification through simulation to output traces.

TABLE I: Simulator comparison. ncsim uniquely combines DAG-native scheduling with CSMA/CA and hidden terminal modeling at flow-level granularity.

Tool	DAG	WiFi	CSMA	Hid. Term.	Pkt-Level	Sweep
ns-3 [2]	×	✓	✓	✓	✓	High
OMNeT++ [3]	×	✓	✓	✓	✓	High
SimGrid [4]	✓	×	×	×	×	Low
WRENCH [5]	✓	×	×	×	×	Low
CloudSim [6]	✓	×	×	×	×	Low
iFogSim [7]	✓	×	×	×	×	Med
EdgeCloudSim [8]	×	✓	×	×	×	Med
ncsim	✓	✓	✓	✓	×	Low

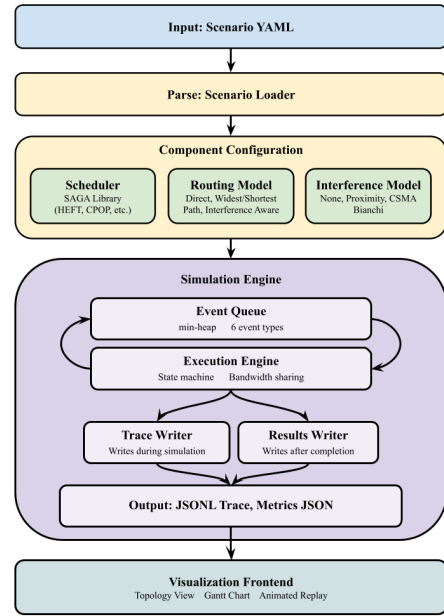


Fig. 1: High-level architecture of ncsim.

A. Design Goals

The design of ncsim is guided by four principles.

Determinism. Given the same scenario and random seed, ncsim produces bit-identical results across runs and platforms by rounding all simulation times to microsecond precision (10^{-6} s), preventing platform-dependent rounding errors from reordering near-simultaneous events.

Modularity. Six abstract base classes define clean extension points (Table III), ensuring that swapping a scheduler does not require changes to the network model, and vice versa.

Simplicity. Scenarios are specified in declarative YAML files. The entire simulator is approximately 5,500 lines of Python with minimal dependencies. Example scenario files are available in the project repository.²

²<https://github.com/ANRGUSC/ncsim>

TABLE II: Scope of validity: phenomena modeled by `ncsim` and their impact on scheduling studies.

Phenomenon	Status	Impact
CSMA/CA contention	Modeled (Bianchi)	High
Hidden terminals	Modeled (SINR)	High
MCS rate adaptation	Modeled (11n/ac/ax)	Medium
Path loss	Modeled (log-distance)	High
Pkt-level dynamics	Not modeled	Low [†]
TCP slow-start	Not modeled	Medium [‡]
Fading / mobility	Optional (log-normal)	Low [§]
Multi-slot parallelism	Not modeled	App-dependent

[†]For bulk transfers. [‡]For short flows. [§]For static deployments.

TABLE III: Pluggable abstractions in `ncsim`.

ABC	Implementations
Scheduler	Manual, RoundRobin, HEFT, CPOP
RoutingModel	Direct, WidestPath, ShortestPath
InterferenceModel	None, CSMA Bianchi
LinkModel	Static (future: position-based, TDMA)
QueueModel	FIFO (future: priority, multi-slot)
DAGSource	Single, Multi

Physical grounding. Wireless link bandwidths are derived from RF parameters (transmit power, frequency, path loss exponent) rather than specified as arbitrary constants, ensuring that link rates are physically consistent with node positions.

B. Scope and Assumptions

`ncsim` targets macroscopic scheduling decisions rather than protocol engineering. It intentionally omits packet-level simulation, physical-layer channel dynamics (fading, OFDM), mobility, and multi-slot compute parallelism, retaining the phenomena that most affect scheduling: CSMA/CA contention, hidden terminals, and MCS rate adaptation. Table II summarizes the modeling scope.

The flow-level abstraction is appropriate for the workloads studied here (10 MB payloads per edge), where steady-state throughput dominates and TCP slow-start transients are negligible; for short transfers (sub-100 KB), packet-level simulation would be warranted. The Bianchi model assumes saturation conditions, and the hidden terminal model uses a capture-threshold rather than full capture effects. Together, these assumptions make `ncsim`'s interference-aware makespans conservative upper bounds, particularly in the hidden terminal regime. We use deterministic path loss throughout to isolate the effects of contention and hidden terminals from fading and mobility.

C. Key Abstractions

D. Visualization and Tooling

`ncsim` includes a React/D3 web-based visualization frontend with three views: a **network topology view** showing nodes, links, and active transfers; a **Gantt chart** of task execution and transfer timelines per node; and an **animated replay** with playback controls for observing bandwidth recalculation cascades. Simulation output is written in JSONL trace format (one JSON object per event) for integration with external

analysis tools. The command-line interface supports batch execution, facilitating the automated parameter sweeps used throughout Section VII.

IV. SYSTEM MODEL

In this section, we present the formal models underlying `ncsim`: the network and compute infrastructure, the DAG workflow representation, the scheduling and routing frameworks, and the physically-grounded 802.11 WiFi model that captures contention and hidden terminal effects.

A. Network Model

A network consists of *nodes* and directed *links*. Each node v has a compute capacity C_v (compute units per second) and an optional 2D position (x_v, y_v) used for RF calculations. Each link ℓ from node u to node v has a bandwidth B_ℓ (MB/s) and latency L_ℓ (seconds). Topologies are specified in YAML and can be arbitrary directed graphs.

B. Compute Model

A task t has a compute cost w_t (in compute units). The execution time on node v is w_t/C_v . Each node processes one task at a time using FIFO queuing. Tasks follow a five-state lifecycle: PENDING $\rightarrow$ READY $\rightarrow$ QUEUED $\rightarrow$ RUNNING $\rightarrow$ COMPLETED.

C. DAG Workflow Model

Workflows are modeled as directed acyclic graphs (DAGs) where nodes represent tasks and edges represent data dependencies. Each edge $e = (t_i, t_j)$ carries a data payload of size d_e (in MB) that must be transferred from the node executing t_i to the node executing t_j before t_j can begin. If t_i and t_j are placed on the same node, the transfer is instantaneous (zero cost). DAGs are injected at specified simulation times.

D. Scheduling Framework

The scheduler receives a `NetworkSnapshot`, an immutable view of node capacities, link bandwidths, queue depths, and active transfers, and returns a `PlacementPlan` mapping each task to a node. `ncsim` includes four schedulers:

- **Manual:** Uses `pinned_to` annotations from the YAML.
- **RoundRobin:** Cycles through nodes, respecting pin constraints.
- **HEFT/CPop:** Adapted from the SAGA scheduling library [10] via an adapter that constructs a fully-connected virtual network with actual bandwidths for connected pairs and near-zero bandwidth for unreachable pairs, causing the scheduler to naturally avoid infeasible placements.

E. Routing and Bandwidth Sharing

Three routing models determine the path for data transfers:

- **Direct:** Only allows transfers over explicit single-hop links.
- **WidestPath:** Modified Dijkstra maximizing bottleneck bandwidth.
- **ShortestPath:** Standard Dijkstra minimizing total latency.

Multi-hop transfers use store-and-forward semantics: the effective bandwidth is the minimum (bottleneck) across all links in the path, and latency is the sum. When multiple transfers share a link, each receives a fair share:

$$B_{\text{per-flow}}(\ell) = \frac{B_\ell \cdot f(\ell)}{N_\ell} \quad (1)$$

where $f(\ell)$ is the interference factor and N_ℓ is the number of concurrent flows on link ℓ . When a transfer starts or completes, all affected transfers are recalculated and their completion events rescheduled.

F. WiFi / 802.11 Model

The WiFi model replaces static link bandwidths with rates derived from RF parameters, and models contention and hidden terminal interference using physically-grounded analytical models. The physical-layer and MAC models draw on standard 802.11 analysis: path loss and MCS selection are standard; carrier-sensing range and conflict graph construction are adapted from the literature; the Bianchi contention model and capture-threshold hidden terminal factor are our specific contributions to this simulation framework. The following subsections formalize these mechanisms.

1) *Physical Layer*: The received power at distance d uses log-distance path loss:

$$\text{PL}(d) = \text{PL}(d_0) + 10n \log_{10}\left(\frac{d}{d_0}\right) \quad (2)$$

$$P_{\text{rx}}(d) = P_{\text{tx}} - \text{PL}(d) \quad (3)$$

where $\text{PL}(d_0)$ is the Friis free-space loss at reference distance $d_0 = 1$ m, n is the path loss exponent, and P_{tx} is the transmit power. The signal-to-noise ratio is $\text{SNR} = P_{\text{rx}} - N_0$ (in dB), where N_0 is the noise floor. The path loss exponent n ranges from 2 (free-space) to ≥ 3.5 (dense indoor).

The PHY data rate is selected from the appropriate 802.11 MCS table based on the computed SNR:

$$R_\ell = \max\{r_k : \text{SNR}_\ell \geq \text{SNR}_{\text{min},k}\} \quad (4)$$

This selects the highest-rate MCS whose minimum SNR requirement is met, producing a discrete rate staircase, a key feature that continuous-rate models fail to capture.

2) *Carrier Sensing and Conflict Graph*: The carrier sensing range d_{CS} is the maximum distance at which a transmission triggers the clear channel assessment (CCA) mechanism:

$$d_{\text{CS}} = d_0 \cdot 10^{\frac{P_{\text{tx}} - \theta_{\text{CCA}} - \text{PL}(d_0)}{10n}} \quad (5)$$

The conflict graph $G_C = (L, E_C)$ is defined over links. Without RTS/CTS, two links conflict if either transmitter can sense any node of the other link. With RTS/CTS, any node of one link sensing any node of the other creates a conflict.

3) *CSMA Bianchi Model* (*csma_bianchi*): The dynamic model separates two interference mechanisms. Let $\mathcal{A}$ be the set of currently active links, $\mathcal{C}_\ell = \mathcal{A} \cap \mathcal{N}(\ell)$ the active contending neighbors, and $\mathcal{H}_\ell = (\mathcal{A} \setminus \mathcal{N}(\ell)) \setminus \{\ell\}$ the active hidden terminals.

a) *Contention factor*: The number of contending stations is $n = 1 + |\mathcal{C}_\ell|$. Using Bianchi's saturation throughput analysis [9], the MAC efficiency $\eta(n)$ is computed from the coupled equations for transmission probability τ and collision probability p :

$$\tau = \frac{2(1-2p)}{(1-2p)(W_{\text{min}}+1) + pW_{\text{min}}(1-(2p)^m)} \quad (6)$$

$$p = 1 - (1-\tau)^{n-1} \quad (7)$$

with minimum contention window $W_{\text{min}} = 16$ and maximum backoff stage $m = 6$. We solve numerically via bisection (Section VI-B) to obtain the equilibrium (τ^*, p^*) . The MAC efficiency is then:

$$\eta(n) = \frac{P_{\text{success}} \cdot T_{\text{success}}}{\mathbb{E}[T_{\text{slot}}]} \quad (8)$$

where P_{success} , T_{success} , and $\mathbb{E}[T_{\text{slot}}]$ are the probability, duration, and expected slot time for successful transmissions, respectively. Each contending station receives a fair share $\eta(n)/n$ of the channel capacity.

b) *Hidden terminal factor*: Hidden terminals transmit simultaneously with link ℓ , causing interference at the receiver. The SINR is:

$$\text{SINR}_\ell = 10 \log_{10} \left(\frac{P_{\text{rx}}^{(\text{lin})}}{N_0^{(\text{lin})} + \sum_{j \in \mathcal{H}_\ell} P_j^{(\text{lin})}} \right) \quad (9)$$

In 802.11, each frame is transmitted at a pre-selected MCS and either decoded successfully or lost entirely; there is no fallback to a lower MCS during reception [32], [33]. The MCS rate-selection thresholds include operating margins (implementation loss, fading margin, AGC settling) that are not required for frame decoding. The *decode threshold* for a given MCS is therefore lower than the selection threshold by a capture margin Δ :

$$\theta_{\text{decode}} = \theta_{\text{select}} - \Delta \quad (10)$$

We use $\Delta = 5$ dB, consistent with the gap between ns-3's `IdealWifiManager` selection and its `TableBasedErrorRateModel` decode thresholds, and with measured capture thresholds of 4–10 dB in the literature [34]. The hidden terminal factor is:

$$f_{\text{HT}}(\ell) = \begin{cases} 1.0 & \text{if } \text{SINR}_\ell \geq \theta_{\text{decode}} \\ 0.01 & \text{otherwise (frame failure)} \end{cases} \quad (11)$$

c) *Combined factor*: The total interference degradation combines both effects multiplicatively:

$$f(\ell) = f_{\text{HT}}(\ell) \cdot \frac{\eta(n)}{n} \quad (12)$$

clamped to $[0.01, 1.0]$. When transfers start or complete, `ncsim` recalculates $f(\ell)$ for all active transfers sharing a conflict graph neighbor with the affected link, and reschedules their completion events accordingly.

V. SIMULATION ENGINE

In this section, we describe the discrete-event simulation engine that forms the core of `ncsim`. The engine orchestrates the interplay between task execution, data transfer, and interference dynamics through an event-driven architecture.

A. Event Processing

`ncsim` uses an event-driven DES with a min-heap priority queue. Six event types are processed in priority order at each simulation time:

- 1) DAGINJECT: Invokes scheduler, initializes task states
- 2) TASKCOMPLETE: Frees node, triggers output transfers
- 3) TRANSFERCOMPLETE: Delivers data, checks readiness
- 4) TASKREADY: Starts task or enqueues if node busy
- 5) TASKSTART: Begins execution, schedules completion
- 6) TRANSFERSTART: Acquires link bandwidth, schedules completion

The priority ordering among event types at the same simulation time is critical for causal correctness. Completions are processed before starts, and task completions before transfer completions, so that freed nodes can accept new work immediately. Consider a scenario where task T_3 completes on node v at time t , and transfer $X_{2 \rightarrow 4}$ also completes at time t , making task T_4 ready. The ordering ensures that T_3 's completion frees node v before T_4 attempts to start, so T_4 can potentially be placed on the now-idle node v rather than being queued.

Events are cancellable via lazy deletion: when bandwidth recalculation invalidates a previously-scheduled completion time, the stale event is marked invalid and a new event is inserted. Stale events are discarded when they reach the head of the queue. All times are rounded to microsecond precision (10^{-6} s) to ensure deterministic behavior across platforms.

B. Bandwidth Recalculation Cascade

The most technically subtle aspect of the simulation engine is the bandwidth recalculation cascade that occurs whenever a transfer starts or completes. This cascade is the mechanism by which `ncsim` captures the dynamic nature of wireless interference: the effective bandwidth of a link depends on which other links are simultaneously active, and this set changes throughout the simulation as transfers begin and end.

When a transfer starts or completes on link ℓ , the engine identifies all active transfers whose links share a conflict graph neighbor with ℓ . For each affected transfer, the engine performs three steps: (1) recompute the number of contending neighbors $|\mathcal{C}_\ell|$ and hidden terminals $|\mathcal{H}_\ell|$; (2) recalculate the combined interference factor $f(\ell)$ from Equation (12); and (3) update the bottleneck bandwidth for the end-to-end path. If the new bandwidth differs from the previously assumed value, the engine cancels the stale completion event and schedules a new one based on the remaining data and updated bandwidth.

To illustrate, consider a 3×3 grid where transfer X_1 is active on link $(0,0) \rightarrow (1,0)$ with an expected completion at $t = 5.0$ s. At $t = 2.0$ s, transfer X_2 starts on the adjacent link

$(1,0) \rightarrow (2,0)$, which is a conflict graph neighbor of X_1 's link. The engine recalculates: X_1 now has one contending neighbor, so its interference factor drops from $f = 1.0$ (solo) to $f = \eta(2)/2 \approx 0.44$. With approximately 56% less effective bandwidth, the remaining data will take longer to transfer, so the engine cancels the $t = 5.0$ s completion event and schedules a new one at approximately $t = 8.8$ s. This recalculation ripples to any other active transfers that neighbor X_2 's link, potentially triggering further reschedules.

In practice, the cascade is bounded by the conflict graph neighborhood size, which ranges from 4 to 12 for our grid topologies. The overall complexity per simulation is $O(E \cdot K \cdot \log N)$ where E is the total number of events, K is the average neighborhood size, and N is the maximum number of pending events. Each simulation in our 108-run study completes in under 2 seconds on commodity hardware.

VI. VALIDATION

In this section, we validate the correctness of `ncsim`'s WiFi model through three complementary approaches. First, we verify internal self-consistency by comparing simulation output against analytical predictions computed directly from the RF and MAC equations across nine controlled experiments. Second, we validate externally by reproducing the numerical results from Bianchi's seminal paper on 802.11 DCF analysis [9], using his exact parameters. Third, we cross-validate against ns-3 [2] packet-level simulations to confirm that the combined model produces realistic throughput predictions. Together, these validations establish that `ncsim`'s interference model is self-consistent, faithful to the published analytical framework, and agrees with packet-level ground truth.

A. Internal Verification

Nine controlled experiments, each isolating a specific WiFi model component, verify that `ncsim`'s output matches analytical predictions exactly (zero error to the precision reported). These cover MCS rate selection across distances, parallel-link contention and hidden terminal regimes, shared-receiver topologies, staggered transfers with dynamic recalculation, multi-flow bandwidth sharing, five-link hidden terminal cascades, and combined interference scenarios. We present three representative experiments in detail; the N-way contention scaling test appears in Section VI-B.

1) *Experiment 1: Link Length vs. Data Rate:* A single link between two nodes at varying distances from 1 m to 140 m. Table IV shows selected results. The simulated rate matches the analytically predicted MCS-selected rate at every distance. At 140 m the SNR (4.19 dB) falls below MCS 0 (5 dB threshold), correctly yielding zero rate. Figure 2 plots the characteristic staircase pattern: rate is constant within each MCS band and drops discretely at each SNR threshold.

2) *Experiment 2: Parallel Link Separation:* Two parallel 30 m links separated vertically by distances from 5 m to 200 m. Figure 3 illustrates the three interference regimes. At separation ≤ 70 m, all node pairs are within the carrier sensing range (71.2 m), so the links *contend* via Bianchi's model: each gets

TABLE IV: Experiment 1: Link length vs. data rate (802.11ax, 5 GHz).

Dist(m)	SNR(dB)	MCS	Pred(MB/s)	Sim(MB/s)
1	68.58	11	17.925	17.925
12	36.20	9	14.338	14.338
30	24.27	5	8.600	8.600
50	17.61	3	4.300	4.300
75	12.33	2	3.225	3.225
105	7.94	0	1.075	1.075
140	4.19	n/a	0.000	0.000

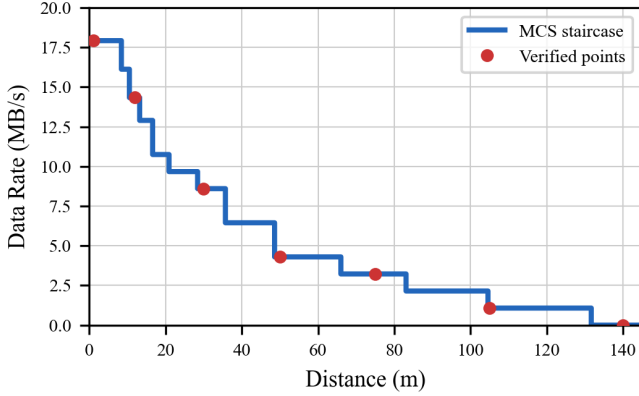


Fig. 2: Experiment 1: PHY data rate vs. link distance showing discrete MCS transitions (802.11ax, 5 GHz, $n=3$). Red dots mark experimentally verified data points from Table IV.

$\eta(2)/2 \approx 0.44$ of the base rate $8.6 \text{ MB/s} = 3.79 \text{ MB/s}$. At separation $\geq 75 \text{ m}$, the links become hidden terminals, and SINR degradation determines the effective rate.

Notably, the effective rate *drops* at the CS range boundary: from 3.79 MB/s (contention) to 3.23 MB/s (hidden terminal at 75 m), before recovering at larger separations. This non-monotonic behavior is the classic *hidden terminal effect*: within sensing range, links coordinate via CSMA/CA and time-share the channel fairly (each receiving 44% of the full 8.6 MB/s); just outside sensing range, links transmit simultaneously without coordination, and the resulting mutual interference degrades each receiver's SINR enough to force a lower MCS rate (from MCS 5 to MCS 2). The hidden terminal regime is thus *worse* than fair contention sharing at close range. As separation increases further, interference power diminishes: SINR recovers to MCS 3 (4.3 MB/s) at 90 m and MCS 4 (6.45 MB/s) at 130 m . All 15 data points match predictions exactly.

3) *Experiment 4: Three Parallel Links*: This experiment extends the two-link topology of Experiment 2 to three parallel 30 m links placed at $y = 0$, $y = s$, and $y = 2s$ (links A, B, C), each carrying a 10 MB transfer. Figure 4 illustrates the topology with separation s as the independent variable.

As s increases, three interference regimes emerge: (i) *all-conflict* ($s \leq 35.6 \text{ m}$): all three pairs are within d_{CS} , each link gets $R_{\text{base}} \cdot \eta(3)/3$; (ii) *mixed* ($35.6 < s \leq 71.2 \text{ m}$): adjacent pairs (A–B, B–C) contend via Bianchi, while the outer pair (A–C at distance $2s$) are hidden terminals, creating asymmetric

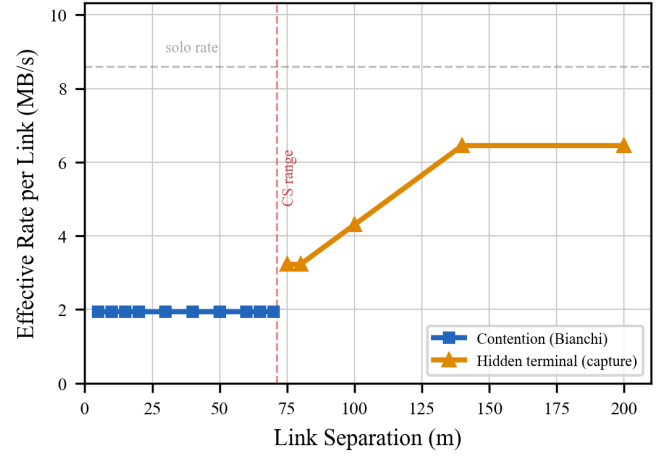


Fig. 3: Experiment 2: Effective per-link rate vs. separation for two parallel 30 m links. Below the CS range (71.2 m), links contend via Bianchi's model. Above it, hidden terminal SINR degradation causes a non-monotonic *dip*: the rate drops from 3.79 to 3.23 MB/s at the boundary because uncoordinated simultaneous transmissions degrade SINR more than fair time-sharing reduces throughput. Rate recovers as interference power falls with increasing distance.

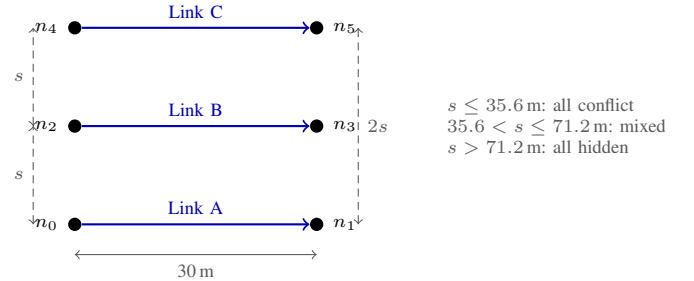


Fig. 4: Experiment 4 topology: three parallel 30 m links at variable separation s . As s increases, the interference regime transitions from all-conflict to mixed (adjacent pairs contend, outer pair A–C are hidden terminals) to all-hidden.

rates; and (iii) *all-hidden* ($s > 71.2 \text{ m}$): no conflict-graph edges, each link sees the others purely as hidden terminals. Because A/C and B generally have different rates, one group finishes first, and the remaining links' interference conditions change. The analytical prediction accounts for this multi-phase behavior.

Table V shows the results. All 11 test points match exactly. In the mixed regime at $s=70 \text{ m}$, an interesting crossover occurs where A/C become faster than B, because the outer links benefit from their mutual hidden-terminal distance while B contends with both neighbors. This experiment validates the model's ability to correctly compose Bianchi contention and SINR degradation in a single topology with asymmetric interference conditions and multi-phase dynamic recalculation.

4) *Additional Validation Experiments*: Five additional experiments validate: (3) two-transmitter shared-receiver topologies,

TABLE V: Experiment 4: Three parallel links at varying separation. In the mixed regime the middle link B contends with both neighbors while outer links A, C contend only with B and see each other as hidden terminals.

$s(m)$	Regime	Pred R_A	Sim R_A	Pred R_B	Sim R_B
10	all-conf	2.461	2.461	2.461	2.461
35	all-conf	2.461	2.461	2.461	2.461
40	mixed	1.861	1.861	2.461	2.461
50	mixed	2.174	2.174	2.461	2.461
70	mixed	2.840	2.840	2.720	2.720
75	all-hid	3.225	3.225	2.867	2.867
100	all-hid	4.300	4.300	3.822	3.822
150	all-hid	6.450	6.450	5.160	5.160

A=C by symmetry in all cases. Rates in MB/s.

TABLE VI: Reproduction of Bianchi [9] Table III: normalized saturation throughput S for $W=32$, $m=3$.

n	Computed	Published	Rel. Error
2	0.847311	0.8473	0.001%
3	0.836828	0.8368	0.003%

(5) staggered transfer starts with dynamic recalculation, (6) per-link bandwidth sharing with multiple flows, (8) five-link hidden terminal cascades with different data sizes, and (9) combined bandwidth sharing with hidden terminal interference. All pass.

B. External Validation Against Bianchi (2000)

Internal validation confirms self-consistency but does not establish that our implementation of Bianchi's equations is correct. We therefore reproduce the numerical results from Bianchi's original paper [9] using his exact parameters: FHSS 1 Mbps basic access (no RTS/CTS), with slot duration $50 \mu s$, SIFS $28 \mu s$, DIFS $128 \mu s$, propagation delay $1 \mu s$, payload 8184 bits, MAC header 272 bits, PHY header 128 bits, and ACK 112 bits.

1) *Table III Reproduction:* Table VI compares our computed normalized saturation throughput S against the values published in Bianchi's Table III for $W=32$, $m=3$. At $n=2$, our solver computes $S = 0.847311$ versus the published 0.8473 (relative error 0.001%). At $n=3$, we compute $S = 0.836828$ versus 0.8368 (relative error 0.003%). These sub-basis-point differences are consistent with rounding in the published table.

Our solver for the coupled equations (Equations (6) and (7)) uses a bisection method rather than simple fixed-point iteration. Simple iteration can oscillate when the contention window is small or the number of stations is large, because the mapping can have a slope exceeding unity near the fixed point. Bisection is guaranteed to converge within $\lceil \log_2(1/\epsilon) \rceil$ iterations for any tolerance ϵ . We use $\epsilon = 10^{-12}$, requiring at most 40 bisection steps.

2) *Figure 6 Reproduction:* Figure 5 reproduces Bianchi's Figure 6 throughput curves for $W=32$, $m=5$ and $W=128$, $m=3$ across $n=5$ to $n=50$ stations. Both curves match the expected monotonic throughput decay; the $W=128$ curve reproduces the slight non-monotonicity between $n=5$ and $n=10$ consistent with Bianchi's published results. Data points

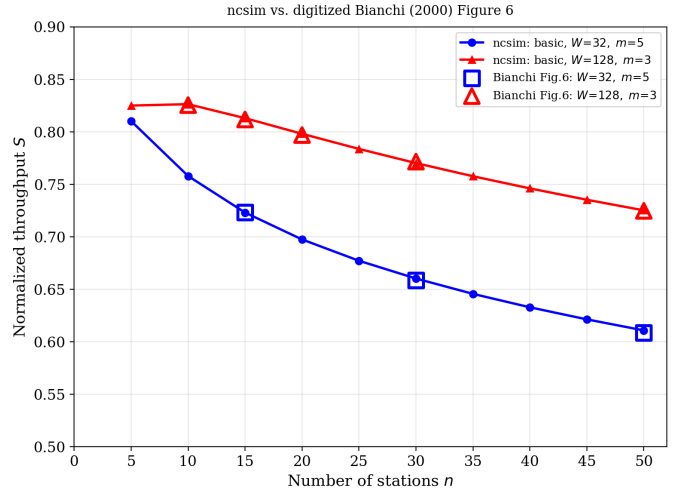


Fig. 5: Reproduction of Bianchi [9] Figure 6: normalized saturation throughput vs. number of stations for two (W, m) configurations. The $W=128$ curve's non-monotonicity between $n=5$ and $n=10$ is consistent with Bianchi's published results. Digitized data points overlay our curves to within 0.34% relative error, confirming faithful reproduction.

TABLE VII: N-way Bianchi contention scaling (30 m links at 5 m separation). $\eta(n)$ is the Bianchi MAC efficiency.

n	$\eta(n)$	$\eta(n)/n$	Pred(MB/s)	Sim(MB/s)
2	0.881	0.440	3.787	3.787
3	0.859	0.286	2.461	2.461
4	0.837	0.209	1.799	1.799
5	0.818	0.164	1.407	1.407
6	0.802	0.134	1.149	1.149
7	0.788	0.113	0.968	0.968
8	0.726	0.091	0.780	0.780

digitized from Bianchi's original figure overlay our computed curves to within 0.34% relative error.

3) *N-Way Contention Scaling:* To confirm that the Bianchi model scales correctly within the full simulation loop, we place $n \in \{2, \dots, 8\}$ parallel 30 m links at 5 m separation (all within carrier-sensing range), each carrying a 10 MB transfer.

Table VII shows all seven data points match exactly, confirming that the Bianchi solver produces correct throughput predictions when embedded in the full simulation engine with real RF parameters and dynamic recalculation.

C. Cross-Validation Against ns-3

The preceding validation establishes internal self-consistency and faithful reproduction of Bianchi's analytical results, but all comparisons are between analytical formulas; ncsim implements the same equations it validates against. To test whether the *combined* model behavior (path loss + MCS selection + CSMA/CA contention + hidden terminal capture effects) produces realistic throughput predictions, we cross-validate against ns-3 [2], a widely-used packet-level network simulator that models 802.11 at full protocol fidelity.

Both experiments use 802.11ax at 5 GHz on a 20 MHz channel with ConstantRateWifiManager at HE-MCS 5 (64-QAM, coding rate 2/3), TX power 20 dBm, CCA threshold -82 dBm, noise figure 6 dB (effective floor -95 dBm), $CW_{\min}=16$, $CW_{\max}=1024$, guard interval 3200 ns, 1 spatial stream, RTS/CTS off, and, critically, A-MPDU disabled to match Bianchi’s single-frame assumption. The propagation model is log-distance with exponent 3.0 and Friis reference loss 46.4 dB at 1 m, identical to *ncsim*’s `RFCConfig` defaults. Each ns-3 point represents the mean over 20 independent seeds with 30 s simulation time (first 2 s discarded as warmup).

1) *Experiment 1: Contention Scaling*: We place $n \in \{1, \dots, 8\}$ co-located STA-AP link pairs at 30 m link length and 5 m vertical separation, all well within the 71.2 m carrier-sensing range, creating a complete conflict graph. Each STA sends saturated UDP traffic to its AP. Figure 6(a) overlays *ncsim*’s Bianchi prediction $R_{\text{base}} \cdot \eta(n)/n$ against the ns-3 per-station goodput.

ncsim’s Bianchi model, which accounts for PHY overhead (preamble, SIFS, ACK) in the efficiency computation, tracks ns-3 closely across all station counts. At $n=1$, the solo goodput predictions differ by only 0.3% (3.78 vs. 3.77 MB/s). The maximum error of 7.6% occurs at $n=7$, where the Bianchi fixed-point approximation slightly overestimates MAC efficiency. The mean error across all n is 4.0%. Both curves exhibit the same monotonic decay shape with Pearson correlation $r > 0.999$. All 95% confidence intervals are below ± 0.02 MB/s, confirming that 30 s of saturated traffic reaches steady state.

2) *Experiment 2: Separation Sweep*: Two parallel 30 m links are placed at vertical separation $s \in \{10, 20, \dots, 200\}$ m. As s increases beyond the 71.2 m carrier-sensing boundary, the links transition from Bianchi contention (symmetric time-sharing at $\eta(2)/2$) to the hidden terminal regime, where simultaneous transmissions may corrupt frames at the receiver. *ncsim*’s capture-threshold model (Equation (10)) predicts a *throughput dip* at the CS boundary: per-link rate drops from 1.94 MB/s (contention) to 1.50 MB/s (hidden terminal frame loss) before recovering sharply at $s = 120$ m when the interferer’s power falls below the capture threshold and frames are decoded successfully at the full solo rate (3.78 MB/s). Figure 6(b) overlays ns-3 results.

ns-3 confirms the qualitative behavior across all regimes. In the contention zone ($s \leq 70$ m), ns-3 goodput is stable at 1.92–2.06 MB/s (1–6% from *ncsim*). At the CS boundary ($s = 72$ m), ns-3 exhibits a throughput dip to 1.34 MB/s, confirming *ncsim*’s predicted dip direction and location (11% error). In the hidden terminal zone ($s = 72$ –100 m), *ncsim*’s capture-threshold model predicts 1.50 MB/s and ns-3 measures 1.34–1.50 MB/s (0.5–11% error). At large separations ($s \geq 120$ m) where interference power falls below the capture threshold, both models show recovery: *ncsim* predicts the full solo rate of 3.78 MB/s and ns-3 reaches 3.62–3.77 MB/s (0.3–4.4% error).

The ns-3 simulation scripts, Dockerfile for reproducible builds, and all raw results are included in the paper’s supplementary repository under `paper/ns3/`.

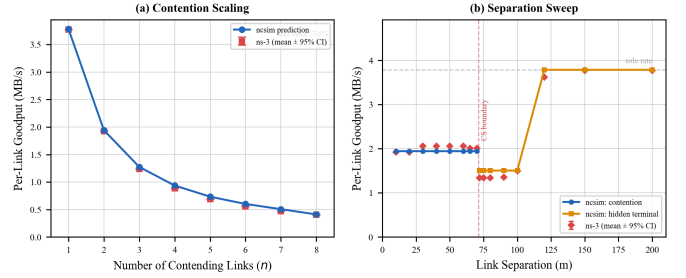


Fig. 6: Cross-validation of *ncsim* against ns-3 packet-level simulation. (a) Per-station saturated goodput under n -way contention: *ncsim* tracks ns-3 within 0.3–7.6% across all station counts ($r > 0.999$). (b) Two-link throughput vs. separation: ns-3 confirms the throughput dip at the CS boundary (71.2 m, dashed red) and recovery at large separations. The capture-threshold model matches within 0.3–11% across all regimes. All parameters aligned per Table VIII; each ns-3 point: mean $\pm$ 95% CI over 20 seeds.

TABLE VIII: Aligned parameters for ns-3 cross-validation.

Parameter	ncsim	ns-3
TX power	20 dBm	TxPowerStart/End = 20
Frequency	5 GHz	Band 5 GHz, ch.36
Channel width	20 MHz	ChannelSettings
Path loss exp.	3.0	Exponent = 3.0
Ref. loss (1 m)	46.4 dB	ReferenceLoss = 46.4
Noise floor	-95 dBm	RxNoiseFigure = 6
CCA threshold	-82 dBm	CcaEdThreshold = -82
$CW_{\min}$	16	MinCw = 15
$CW_{\max}$	1024	MaxCw = 1023
A-MPDU	disabled	MaxAmpduSize = 0
Guard interval	3200 ns	GuardInterval = 3200

VII. EXPERIMENTAL EVALUATION

The central finding of this paper is that interference-free evaluation produces wrong scheduler rankings in over a quarter of scenarios tested. The four subsections that follow build the evidence progressively: routing interactions with interference (Section VII-B), magnitude of interference impact (Section VII-C), rank inversions in a 108-run factorial (Section VII-D), and sensitivity analyses delineating an *interference danger zone*, the parameter regimes where interference is most consequential (Section VII-E).

All experiments use 802.11ax at 5 GHz with a path loss exponent of $n = 3$ (except where explicitly varied), heterogeneous compute capacities (80–300 units/s), 10 MB data payloads per DAG edge, and deterministic seed 42.

Three DAG structures of increasing complexity are used across all experiments:

- **Small (5 tasks)**: Fork-join, $T_0 \rightarrow \{T_1, T_2, T_3\} \rightarrow T_4$.
- **Medium (10 tasks)**: Diamond, $T_0 \rightarrow \{T_1-T_4\} \rightarrow \{T_5-T_8\} \rightarrow T_9$ with selective cross-links between layers.
- **Large (20 tasks)**: Multi-level pipeline, $T_0 \rightarrow 4 \rightarrow 6 \rightarrow 6 \rightarrow 3$ tasks per layer with sparse inter-layer connections.

TABLE IX: Scheduler inputs. No scheduler observes interference.

Scheduler	Planner Inputs	Network View	Intf?
Manual	Pin annotations	N/A	No
RoundRobin	Node list	N/A	No
HEFT	DAG, C_v , B_ℓ	Full virtual net [†]	No
CPOP	DAG, C_v , B_ℓ	Full virtual net [†]	No

[†]Actual BW for reachable pairs; 0.001 MB/s for unreachable.

All tasks have `compute_cost` = 500 and all edges have `data_size` = 10 MB (except where varied in Section VII-E).

A. Scheduler Information and Fairness

Both the interference-free and interference-aware modes present an identical `NetworkSnapshot` to the scheduler, containing the same WiFi-derived link bandwidths (Table IX). No scheduler sees the conflict graph or contention factors; these are runtime phenomena computed *after* the schedule has been committed. Any difference in makespan is therefore attributable solely to interference that the scheduler could not anticipate.

B. Routing Strategy Comparison

Because multi-hop routing determines contention domains, we first establish how routing strategies interact with interference before evaluating schedulers. We compare widest-path and shortest-path routing under HEFT scheduling with the interference-aware model across grid mesh networks of three sizes (2×2 , 3×3 , 4×4) and three DAG complexities (5, 10, and 20 tasks). All links use 40 m grid spacing with heterogeneous compute capacities (80–300 units/s).

Figure 7 shows the resulting link utilization for each strategy. Link color and thickness indicate the number of bidirectional data flows traversing each link. Widest-path routing funnels nearly all traffic through an 8-hop perimeter corridor (nodes 0–1–2–6–10–14–13–12), loading each link with 9–10 flows and leaving the right half of the network idle. Shortest-path routing distributes traffic across 12 links with shorter routes (avg 3.1 hops vs. 4.5), spreading load more evenly. Under CSMA contention, widest-path’s flow concentration produces severe airtime competition: its makespan is 1341 s compared to 712 s for shortest-path (1.9× slower). The result illustrates that single-flow optimal routing (maximizing bottleneck bandwidth) becomes counterproductive when multiple concurrent transfers compete for shared wireless airtime.

Key findings. Shortest-path routing wins in 7 of 9 configurations with 2 ties. The advantage is most pronounced in the 4×4 grid, where widest-path routing selects longer paths with higher bottleneck bandwidth but incurs substantially more wireless interference due to multi-hop transmissions traversing more contention domains. In the 4×4 grid with 5 tasks, widest-path achieves a makespan of 315.2 s compared to 12.9 s for shortest-path, a 24× difference, because the widest paths route transfers through 3–4 hops across the full grid diameter, creating overlapping contention domains that reduce effective bandwidth to a fraction of the solo rate.

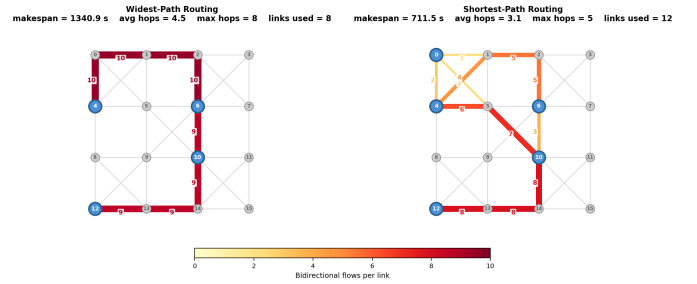


Fig. 7: Link utilization under widest-path (left) vs. shortest-path (right) routing on a 4×4 grid mesh with a 20-task DAG. Widest-path concentrates 9–10 flows on an 8-hop perimeter corridor; shortest-path spreads traffic across 12 links with shorter routes, reducing makespan by 1.9× under CSMA contention.

TABLE X: Routing comparison: makespan (seconds) for widest-path (W) vs. shortest-path (S) under HEFT with `csma_bianchi`.

Network	DAG	W (s)	S (s)	Winner
2×2 (4 nodes)	5 tasks	15.38	15.38	Tie
	10 tasks	32.60	32.50	Shortest
	20 tasks	88.13	64.51	Shortest
3×3 (9 nodes)	5 tasks	19.06	19.06	Tie
	10 tasks	79.88	35.16	Shortest
	20 tasks	142.94	96.70	Shortest
4×4 (16 nodes)	5 tasks	315.20	12.88	Shortest
	10 tasks	1396.9	827.23	Shortest
	20 tasks	1703.4	794.39	Shortest

C. Interference Impact Analysis

We measure the impact of CSMA/CA interference by running each of the nine grid-network scenarios with the interference-aware model versus the interference-free model, both using shortest-path routing and HEFT. As detailed in Section VII-A, both modes present identical `NetworkSnapshot` inputs to the scheduler; the only difference is the presence or absence of runtime contention and hidden terminal effects. Table XI summarizes the results.

Key findings. Interference impact ranges from 35% to over 5400% across all configurations. Several patterns emerge:

- **No configuration escapes contention entirely.** Even the smallest scenario (2×2 , 5 tasks) shows a 35% slowdown, because the Bianchi MAC overhead reduces effective throughput whenever links share the channel.
- **Denser networks amplify interference.** The 3×3 grid (32 links) shows 126–465% slowdown, while the 4×4 grid (68 links) shows up to 5406%. More links create larger conflict neighborhoods.
- **Dense interference creates cascading delays.** In the 4×4 grid, HEFT distributes tasks across 16 nodes assuming uncontested bandwidth. During execution, many simultaneous transfers contend, reducing effective rates and causing HEFT’s schedule to become severely suboptimal.
- **CPOP shows equal or worse slowdowns.** CPOP matches

TABLE XI: Interference impact: makespan (seconds) without interference vs. with `csma_bianchi`, using shortest-path routing. HEFT and CPOP produce identical placements on the 5-task DAG but diverge as DAG complexity grows.

Net.	DAG	HEFT			CPOP		
		None	Bian.	Slow.	None	Bian.	Slow.
2×2	5T	11.43	15.38	+35%	11.43	15.38	+35%
	10T	18.76	32.50	+73%	18.76	41.32	+120%
	20T	27.50	64.51	+135%	29.53	70.96	+140%
3×3	5T	8.43	19.06	+126%	8.43	19.06	+126%
	10T	12.76	35.16	+176%	13.65	52.01	+281%
	20T	17.11	96.70	+465%	19.98	104.24	+422%
4×4	5T	8.93	12.88	+44%	8.93	12.88	+44%
	10T	15.02	827.23	+5406%	15.02	827.23	+5406%
	20T	21.75	794.39	+3553%	20.65	873.26	+4129%

HEFT on the 5-task DAG (identical placements), but exhibits higher slowdowns on larger DAGs in the 2×2 and 3×3 grids (e.g., 281% vs. 176% for 10 tasks on 3×3). CPOP’s critical-path-based allocation concentrates communication along fewer paths, making those paths more vulnerable to contention. In the 4×4 grid, both schedulers suffer extreme slowdowns.

These results demonstrate that ignoring wireless interference in scheduling can lead to order-of-magnitude performance prediction errors, motivating the physically-grounded model in `ncsim`.

D. Scheduler Comparison and Rank Inversions

The interference impact results above suggest a deeper question: does ignoring interference merely produce inaccurate makespan predictions, or does it lead to wrong *choices* among schedulers? To answer this, we execute a full factorial comparison: 3 networks × 3 DAGs × 3 schedulers (HEFT, CPOP, RoundRobin) × 2 routings × 2 interference models = 108 simulation runs. All runs use 802.11ax at 5 GHz, heterogeneous compute capacities, and seed 42.

For each of the 18 (network, DAG, routing) triples, we identify the best scheduler under each interference model. A *rank inversion* occurs when the winning scheduler changes between the interference-free and interference-aware models. The *scheduling regret* is the relative performance penalty from deploying the interference-free-optimal scheduler in the presence of interference.

Table XII presents makespan ratios for each scheduler relative to the best under the interference-aware model; a ratio of 1.00 denotes the optimal choice. The key statistics are:

- **Rank inversion rate: 27.8%** (5 of 18 triples).
- **Mean relative regret: 20.2%** across all 18 triples.
- **Maximum relative regret: 168.3%** (4 × 4 grid, 10-task DAG, widest-path: HEFT chosen under the interference-free model gives 1396.9 s, but round-robin achieves 520.6 s under the interference-aware model).

HEFT dominates under interference-free models. Under the interference-free model, HEFT or CPOP wins in all 18

triples. This is expected: both algorithms optimize the critical path by distributing tasks across nodes to maximize parallelism, assuming uncontested bandwidth. The sophisticated scheduling logic of HEFT and CPOP provides clear advantages when the network behaves as these algorithms assume it will.

Round-robin wins in large networks under interference.

In the 4 × 4 grid with the interference-aware model and widest-path routing, round-robin wins for all three DAG sizes, a striking reversal. The mechanism is clear: HEFT spreads tasks across many of the 16 available nodes to exploit parallelism, but this creates many concurrent multi-hop transfers that contend on shared wireless channels. Round-robin’s simpler placement cycles through nodes in order, which tends to concentrate work on nearby nodes with fewer concurrent transfers and thus less contention.

To illustrate this mechanism concretely, consider the most dramatic rank inversion: the 4 × 4 grid with 10 tasks and widest-path routing. Under the interference-free model, HEFT achieves a makespan of 14.9 s by distributing tasks across many of the 16 nodes, creating multiple concurrent inter-node transfers that exploit the assumed link bandwidths. However, under the interference-aware model, these concurrent transfers contend on overlapping conflict graph neighborhoods: each transfer’s effective bandwidth drops to a fraction of its solo rate as the Bianchi model allocates channel time among contenders. The resulting makespan increases to 1,396.9 s, a 94× blowup. In contrast, round-robin achieves 520.6 s under the same interference model. Although round-robin makes no attempt to optimize the critical path, its placement pattern generates fewer concurrent transfers, resulting in less severe contention. The scheduling regret, the penalty for choosing HEFT based on the interference-free evaluation, is 168%. A researcher evaluating these schedulers without interference modeling would confidently select HEFT, unaware that it produces 2.7× worse performance than a round-robin baseline.

Routing modulates the effect. Shortest-path routing mitigates rank inversions in some cases by reducing the number of hops per transfer and thus the number of contention domains traversed. The 4 × 4 / 5-task scenario shows a rank inversion under widest-path routing (60% regret) but not under shortest-path (zero regret). However, shortest-path does not eliminate inversions entirely: in the 4 × 4 / 10-task scenario, the inversion persists even with shortest-path routing (98% regret), because the DAG is large enough to generate concurrent transfers even along short paths.

Figure 8 visualizes the regret ratio (naive makespan divided by oracle makespan) for all 18 triples. The 13 triples at 1.0 show zero regret; the 5 bars above 1.0 are rank inversions, with the 4 × 4 / 10-task / widest-path case reaching 2.68×.

These results demonstrate a key capability of `ncsim`: by jointly modeling scheduling and wireless interference, it reveals that interference-free evaluations can lead to systematically wrong policy choices, a finding that would be invisible to tools that abstract away wireless contention.

TABLE XII: Makespan ratios under `csma_bianchi` relative to the best scheduler for each (network, DAG, routing) triple. A ratio of 1.00 denotes the optimal choice; higher values show how much worse that scheduler performs. “None↓” indicates which scheduler an interference-free evaluation would select. Bold entries mark rank inversions where the interference-free choice is suboptimal.

Net.	DAG	Widest-path				Shortest-path			
		H	C	RR	None↓	H	C	RR	None↓
2×2	5T	1.00	1.00	2.71	HEFT	1.00	1.00	2.01	HEFT
	10T	1.00	1.52	2.38	HEFT	1.00	1.27	1.74	HEFT
	20T	1.13	1.00	1.63	CPOP	1.00	1.10	1.69	HEFT
3×3	5T	1.00	1.00	2.00	HEFT	1.00	1.00	2.00	HEFT
	10T	1.19	1.00	8.32	CPOP	1.00	1.48	13.05	HEFT
	20T	1.00	1.79	6.49	HEFT	1.00	1.08	8.31	HEFT
4×4	5T	1.60	1.60	1.00	HEFT	1.00	1.00	6.56	HEFT
	10T	2.68	2.68	1.00	HEFT	1.98	1.98	1.00	HEFT
	20T	1.01	1.28	1.00	CPOP	1.00	1.10	1.97	CPOP

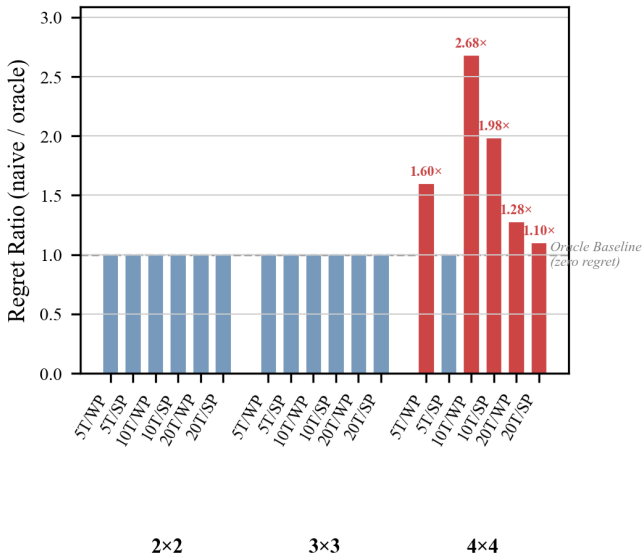


Fig. 8: Scheduling regret ratios. Each bar shows the ratio of the interference-free-optimal makespan (scheduler chosen under the interference-free model, run under the interference-aware model) to the oracle makespan (best scheduler under interference). A ratio of 1.0 indicates zero regret; red bars mark rank inversions where the interference-free choice is suboptimal.

E. Sensitivity Analysis

The results above demonstrate that wireless interference can dramatically affect scheduler rankings. However, a natural question arises: under what conditions is interference most consequential, and when can it safely be ignored? To address this question, we conduct two sensitivity sweeps that vary key parameters affecting interference severity: the communication-to-computation ratio (data size per DAG edge) and the level of concurrent workload (number of simultaneous DAGs).

1) *Communication-to-Computation Ratio*: The communication-to-computation ratio (CCR) determines the relative importance of network transfer time versus

compute time. We sweep data size per edge across $\{1, 2, 5, 10, 20, 50, 100\}$ MB on the 3×3 grid with HEFT and shortest-path routing, testing all three DAG types (5, 10, and 20 tasks). Figure 9 shows the interference slowdown as a function of CCR.

The results reveal a non-monotonic pattern: slowdown peaks at intermediate CCR values (≈ 0.3 – 1.1) and decreases at both extremes. The mechanism differs at each end. At low CCR (1–2 MB), transfers complete so quickly that the probability of temporal overlap between concurrent flows is small, and contention has little opportunity to develop. At very high CCR (50–100 MB), HEFT’s scheduling logic recognizes the enormous transfer costs and collapses tasks onto fewer nodes to avoid inter-node communication entirely, a scheduling adaptation that eliminates contention by eliminating transfers. The interference “danger zone” lies in between, where transfers are long enough to overlap but the scheduler still distributes tasks across multiple nodes. For the 20-task DAG, the peak slowdown of $7.3\times$ occurs at 10 MB per edge, corresponding to a CCR of approximately 0.5 (where transfer time and compute time are comparable).

2) *Multi-DAG Contention*: In practice, edge computing platforms rarely service a single workflow in isolation; multiple applications submit DAGs concurrently, and their transfers share the same wireless medium. To evaluate how `ncsim` captures this multi-tenant scenario, we inject $k \in \{1, 2, 3, 4, 5\}$ identical 5-task fork-join DAGs with 0.5 s stagger on the 3×3 grid (HEFT, shortest-path). Each DAG is scheduled independently, meaning that HEFT optimizes each workflow without knowledge of the others’ transfers.

Figure 10 shows that contention scaling is super-linear under `csma_bianchi`. Without interference, $k=5$ DAGs produce $2.14\times$ the makespan of $k=1$ (18.0 s vs. 8.4 s), reflecting the expected linear increase from staggered injection plus queuing at compute nodes. With interference, however, $k=5$ produces $3.86\times$ the single-DAG makespan (73.5 s vs. 19.1 s). The gap between the two curves widens with each additional DAG: the slowdown factor grows from $2.26\times$ at $k=1$ to $2.98\times$ at $k=2$ to $4.08\times$ at $k=5$, indicating a positive feedback loop. Each additional DAG contributes not only its own transfers to

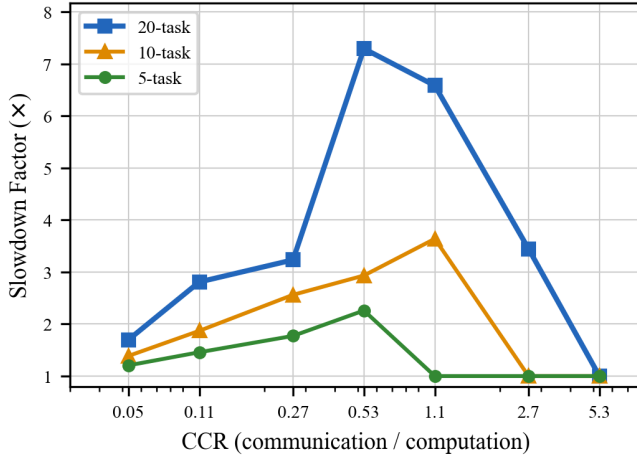


Fig. 9: Interference slowdown vs. CCR (3×3 grid, HEFT, shortest-path). Slowdown peaks at intermediate CCR values where transfers are long enough to contend but not so large that the scheduler collapses tasks onto one node. The 20-task DAG reaches $7.3\times$ at $CCR \approx 0.53$.

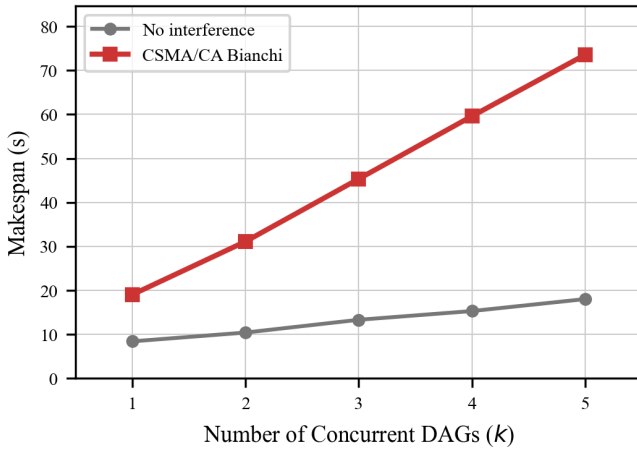


Fig. 10: Multi-DAG makespan scaling on 3×3 grid (5-task DAGs, HEFT, shortest-path). Contention amplifies super-linearly: the gap between the curves widens from $2.26\times$ ($k=1$) to $4.08\times$ ($k=5$).

the contention pool but also prolongs the duration of existing transfers (through reduced effective bandwidth), which in turn increases the temporal overlap with subsequent DAGs. This super-linear scaling is invisible to interference-free models, which predict near-linear growth and would substantially underestimate the makespan of multi-tenant edge deployments.

3) *Sensitivity Summary*: Table XIII consolidates the findings across both sensitivity dimensions. Taken together, these results delineate an *interference danger zone*, the conjunction of parameter settings under which interference effects are most severe and most likely to change scheduling conclusions. Wireless interference has the greatest impact, and is most likely

TABLE XIII: Sensitivity analysis summary. Each row identifies the parameter regime where interference impact is most severe.

Experiment	Parameter	Danger Zone	Peak Effect
CCR	Data size (MB)	5–20 MB	$7.3\times$ at 10 MB
Multi-DAG	DAG count k	$k \geq 2$	$4.1\times$ gap at $k=5$

TABLE XIV: Scalability: makespan ratios on a 100-node random geometric graph (312 links, avg. degree 6.2) with 30-, 40-, and 50-task DAGs. Format follows Table XII. Interference slowdowns range from $48\times$ to $136\times$.

DAG	Widest-path				Shortest-path			
	H	C	RR	None↓	H	C	RR	None↓
30T	1.24	1.00	2.40	HEFT	1.16	1.00	1.89	HEFT
40T	1.00	1.01	2.04	CPOP	1.00	1.09	2.29	HEFT
50T	1.00	1.59	2.31	HEFT	1.18	1.00	1.46	CPOP

to produce rank inversions, at moderate communication-to-computation ratios (5–20 MB payloads per edge) with multiple concurrent workflows ($k \geq 2$).

These conditions are not exotic. Data payloads of 5–20 MB per DAG edge are representative of computer vision pipelines (transferring image frames between processing stages) and sensor fusion applications (aggregating data from multiple modalities). Multiple concurrent workflows are the norm on shared edge platforms. In other words, the interference danger zone corresponds precisely to the deployment scenarios that motivate edge computing research, which means that tools ignoring interference are most unreliable in exactly the settings where they are most needed.

F. Scalability

To demonstrate that `ncsim`'s findings generalize beyond regular grid topologies, we run a scalability experiment on a 100-node random geometric graph (RGG): 100 nodes placed uniformly in a $500\text{ m} \times 500\text{ m}$ area with links between all pairs within 80 m WiFi range, yielding 312 undirected links (average degree 6.2). We test 30-, 40-, and 50-task multi-level pipeline DAGs across all three schedulers and both routing strategies (36 runs total).

Table XIV presents the makespan ratios. Three of six triples (50%) exhibit rank inversions, an even higher rate than the 27.8% observed on grid topologies. Interference slowdowns are dramatically larger: $48\times$ to $136\times$, compared to the $1.4\times$ – $54\times$ range on grids. The denser conflict neighborhoods in a 100-node RGG create far more contention than a 4×4 grid.

Two patterns differ from the grid results. First, CPOP wins under interference for both 30-task triples (ratios 1.00 vs. HEFT at 1.16–1.24), whereas HEFT dominated the smaller grids. CPOP's critical-path focus yields placements that create fewer concurrent transfers in the dense RGG. Second, round-robin is consistently worst (ratios 1.46–2.40), unlike the 4×4 grid where round-robin sometimes won; with 100 nodes, round-robin's cyclic placement scatters tasks across distant nodes, maximizing multi-hop contention.

The full 36-run experiment completes in under 10 minutes on commodity hardware (single-threaded Python 3.11), confirming that the greedy clique approximation (used automatically for the 312-link conflict graph) keeps `ncsim` tractable at 100+ node scales. The main computational bottlenecks are conflict graph construction (Bron-Kerbosch with greedy fallback for > 50 links) and event queue size (grows with concurrent transfers).

VIII. CONCLUSION AND FUTURE WORK

`ncsim` is a lightweight discrete-event simulator that combines DAG workflow scheduling with physically-grounded 802.11 WiFi models, validated against analytical predictions, Bianchi’s published results, and ns-3 packet-level simulations. It targets macroscopic scheduling decisions rather than protocol engineering, keeping simulation times tractable for large parameter sweeps.

The key finding is that wireless interference causes *systematically wrong scheduler rankings*: the interference-free-optimal scheduler is outperformed under realistic conditions in 27.8% of grid scenarios and 50% of scenarios on a 100-node random geometric graph. Sensitivity analyses show that interference impact peaks at moderate communication-to-computation ratios and amplifies super-linearly with concurrent workflows, precisely the conditions that characterize wireless edge deployments.

These results suggest that interference-aware scheduling, where the scheduler accounts for contention when making placement decisions, is a promising direction. More broadly, evaluations that ignore wireless interference risk systematically wrong conclusions about which scheduling algorithms are superior.

Future work includes: (1) interference-aware scheduling heuristics that query the conflict graph during task placement; (2) dynamic rescheduling that adjusts placements in response to observed congestion; and (3) extending the ns-3 cross-validation to a wider range of traffic patterns and multi-hop topologies.

The `ncsim` simulator and all experimental configurations are available at <https://github.com/ANRGUSC/ncsim>.

ACKNOWLEDGMENT

This work was supported in part by Army Research Laboratory under Cooperative Agreement W911NF-17-2-0196.

REFERENCES

- [1] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, “Edge Computing: Vision and Challenges,” *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, Oct. 2016.
- [2] ns-3 Consortium, “ns-3: A Discrete-Event Network Simulator,” <https://www.nsnam.org/>, accessed Mar. 2026.
- [3] A. Varga and R. Hornig, “An Overview of the OMNeT++ Simulation Environment,” in *Proc. ICST SIMUTools*, 2008.
- [4] H. Casanova, A. Giersch, A. Legrand, M. Quinson, and F. Suter, “Versatile, Scalable, and Accurate Simulation of Distributed Applications and Platforms,” *Journal of Parallel and Distributed Computing*, vol. 74, no. 10, pp. 2899–2917, 2014.
- [5] H. Casanova, S. Pandey, J. Oeth, R. Tanaka, F. Suter, and R. Ferreira da Silva, “WRENCH: A Framework for Simulating Workflow Management Systems,” in *Proc. WORKS Workshop*, 2018, pp. 74–85.
- [6] R. N. Calheiros, R. Ranjan, A. Beloglazov, C. A. F. De Rose, and R. Buyya, “CloudSim: A Toolkit for Modeling and Simulation of Cloud Computing Environments and Evaluation of Resource Provisioning Algorithms,” *Software: Practice and Experience*, vol. 41, no. 1, pp. 23–50, 2011.
- [7] H. Gupta, A. Vahid Dastjerdi, S. K. Ghosh, and R. Buyya, “iFogSim: A Toolkit for Modeling and Simulation of Resource Management Techniques in the Internet of Things, Edge and Fog Computing Environments,” *Software: Practice and Experience*, vol. 47, no. 9, pp. 1275–1296, 2017.
- [8] C. Sonmez, A. Ozgovde, and C. Ersoy, “EdgeCloudSim: An Environment for Performance Evaluation of Edge Computing Systems,” *Transactions on Emerging Telecommunications Technologies*, vol. 29, no. 11, e3493, 2018.
- [9] G. Bianchi, “Performance Analysis of the IEEE 802.11 Distributed Coordination Function,” *IEEE Journal on Selected Areas in Communications*, vol. 18, no. 3, pp. 535–547, Mar. 2000.
- [10] H. Topcuoglu, S. Hariri, and M.-Y. Wu, “Performance-Effective and Low-Complexity Task Scheduling for Heterogeneous Computing,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 13, no. 3, pp. 260–274, 2002.
- [11] T. L. Adam, K. M. Chandy, and J. R. Dickson, “A Comparison of List Schedules for Parallel Processing Systems,” *Communications of the ACM*, vol. 17, no. 12, pp. 685–690, 1974.
- [12] Y.-K. Kwok and I. Ahmad, “Static Scheduling Algorithms for Allocating Directed Task Graphs to Multiprocessors,” *ACM Computing Surveys*, vol. 31, no. 4, pp. 406–471, 1999.
- [13] G. C. Sih and E. A. Lee, “A Compile-Time Scheduling Heuristic for Interconnection-Constrained Heterogeneous Processor Architectures,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 4, no. 2, pp. 175–187, 1993.
- [14] J. Coleman and B. Krishnamachari, “Comparing Task Graph Scheduling Algorithms: An Adversarial Approach,” arXiv preprint arXiv:2403.07120, 2024.
- [15] J. Coleman, B. Krishnamachari, and contributors, “SAGA: Scheduling Algorithms Gathered (software),” <https://github.com/anrgusc/saga>, 2024.
- [16] J. Coleman and B. Krishnamachari, “Parameterized Task Graph Scheduling Algorithm for Comparing Algorithmic Components,” arXiv preprint arXiv:2403.07112, 2024.
- [17] P. Ghosh, Q. Nguyen, P. K. Sakulkar, J. A. Tran, A. Knezevic, J. Wang, Z. Lin, B. Krishnamachari, M. Annavaram, and S. Avestimehr, “Jupiter: A Networked Computing Architecture,” in *Proc. UCC Companion*, 2021.
- [18] A. Poylisher, A. Cichocki, K. Guo, J. Hunziker, L. Kant, B. Krishnamachari, S. Avestimehr, and M. Annavaram, “Tactical Jupiter: Dynamic Scheduling of Dispersed Computations in Tactical MANETs,” in *MILCOM 2021*, 2021.
- [19] X. Zhao, D. Hu, and B. Krishnamachari, “Design and Experimental Evaluation of Algorithms for Optimizing the Throughput of Dispersed Computing,” arXiv preprint arXiv:2112.13875, 2021.
- [20] S. Suryavansh, C. Bothra, K. T. Kim, M. Chiang, C. Peng, and S. Bagchi, “I-BOT: Interference-Based Orchestration of Tasks for Dynamic Unmanaged Edge Computing,” arXiv preprint arXiv:2011.05925, 2020.
- [21] X. Li, M. Abdallah, S. Suryavansh, M. Chiang, K. T. Kim, and S. Bagchi, “DAG-based Task Orchestration for Edge Computing,” in *Proc. SRDS*, 2022, pp. 23–34.
- [22] X. Li, M. Abdallah, Y.-Y. Lou, M. Chiang, K. T. Kim, and S. Bagchi, “Dynamic DAG-Application Scheduling for Multi-Tier Edge Computing in Heterogeneous Networks,” arXiv preprint arXiv:2409.10839, 2024.
- [23] S. Yi, Z. Hao, Q. Zhang, Q. Zhang, W. Shi, and Q. Li, “LAVEA: Latency-aware Video Analytics on Edge Computing Platform,” in *Proc. SEC*, 2017.
- [24] W. Zhang, S. Li, L. Liu, Z. Jia, Y. Zhang, and D. Raychaudhuri, “Hetero-Edge: Orchestration of Real-time Vision Applications on Heterogeneous Edge Clouds,” in *IEEE INFOCOM*, 2019, pp. 1270–1278.
- [25] L. Lin, P. Li, J. Xiong, and M. Lin, “Distributed and Application-aware Task Scheduling in Edge-clouds,” arXiv preprint arXiv:1902.04362, 2019.
- [26] J. Liang, K. Li, C. Liu, and K. Li, “Joint offloading and scheduling decisions for DAG applications in mobile edge computing,” *Neurocomputing*, vol. 424, pp. 160–171, 2021.
- [27] K. Li, “Energy-Constrained DAG Scheduling on Edge and Cloud Servers with Overlapped Communication and Computation,” *Journal of Grid Computing*, 2024.
- [28] L. Long, Z. Liu, J. Shen, and Y. Jiang, “SecDS: A security-aware DAG task scheduling strategy for edge computing,” *Future Generation Computer Systems*, p. 107627, 2025.

- [29] M. Chiang and T. Zhang, "Fog and IoT: An Overview of Research Opportunities," *IEEE Internet of Things Journal*, vol. 3, no. 6, pp. 854–864, 2016.
- [30] M. Chiang, S. Ha, I. Chih-Lin, F. Risso, and T. Zhang, "Clarifying Fog Computing and Networking: 10 Questions and Answers," *IEEE Communications Magazine*, vol. 55, no. 4, pp. 18–20, 2017.
- [31] K. T. Kim, C. Joe-Wong, and M. Chiang, "Coded Edge Computing," in *IEEE INFOCOM*, 2020, pp. 237–246.
- [32] F. Daneshgaran, M. Laddomada, F. Mesiti, M. Mondin, and M. Zanolò, "Saturation Throughput Analysis of IEEE 802.11 in the Presence of Non Ideal Transmission Channel and Capture Effects," *IEEE Trans. Communications*, vol. 56, no. 7, pp. 1167–1178, Jul. 2008.
- [33] M. Zorzi and R. R. Rao, "Capture and Retransmission Control in Mobile Radio," *IEEE J. Sel. Areas Commun.*, vol. 12, no. 8, pp. 1289–1298, Oct. 1994.
- [34] Z. Hadzi-Velkov and B. Spasenovski, "Capture Effect in IEEE 802.11 Basic Service Area Under Influence of Rayleigh Fading and Near/Far Effect," in *IEEE PIMRC*, 2002.